Agente inteligente para la gestión de citas veterinarias

Implementación y comparación de BFS, DFS y A*

Autor: SP. John Alexander Niño Gonzalez

Fecha: 3 de noviembre de 2026

Repositorio:

https://github.com/USUARIO/agente-citas-veterinarias-v7-3

IMPORTANTE: antes de entregar, sustituir USUARIO por el usuario real de GitHub y volver a generar este PDF.

Resumen. Se desarrolló un agente en Python para reservar, cancelar y reprogramar citas veterinarias. El sistema integra persistencia en Google Drive, notificaciones SMTP, consulta administrativa y análisis comparativo de búsqueda no informada e informada. BFS, DFS y A* operan sobre un espacio discreto de fecha, franja horaria y profesional. La solución registra nodos expandidos, nodos generados, tiempo y costo, lo que permite evaluar el comportamiento algorítmico y mantener trazabilidad de las operaciones.

1. Problema y arquitectura

El agente recibe solicitudes desde el portal de usuario y desde la interfaz administrativa. Una reserva confirmada puede cancelarse o reprogramarse. En una reprogramación, los algoritmos buscan una alternativa disponible; en una cancelación, analizan el espacio liberado y detectan la cita activa más cercana sin reasignarla automáticamente. Los resultados y el historial se conservan en un archivo JSON de Google Drive.

La solución se distribuye como notebook ejecutable en Google Colab y como módulo Python independiente. El repositorio contiene código fuente, pruebas, benchmark, documentación y archivo de dependencias. Las credenciales SMTP se almacenan en Secrets de Colab y no deben publicarse.

2. Formalización del espacio de estados

El problema se representa mediante la tupla s = (d, h, v), donde d es el desplazamiento en días respecto de la fecha solicitada, h es el índice de una de siete franjas horarias y v identifica uno de tres profesionales. Para un horizonte de 45 días, el límite superior es 46 x 7 x 3 = 966 estados.

Componente	Formalización
Estado inicial	s0 = (0, hora solicitada, profesional actual)
Acciones	Avanzar/retroceder día, hora o profesional
Transición	Aplicar una acción si el estado permanece dentro del dominio
Prueba de meta	El estado pertenece al conjunto de citas disponibles
Costo	Día = 2; hora = 1; profesional = 1,5
Heurística A*	$h(s)=2\|\Delta d\|+\|\Delta h\|+1{,}5\|\Delta v\|$ hacia la meta más cercana

3. Implementación de los algoritmos

BFS. Utiliza una cola FIFO y explora por niveles. Es completo en espacios finitos y encuentra la solución de menor profundidad cuando los pasos tienen costo uniforme. En este proyecto sirve como línea base sistemática.

DFS. Utiliza una pila LIFO y profundiza antes de retroceder. Requiere menos memoria, pero puede recorrer rutas extensas y no garantiza la alternativa de menor costo. El dominio está acotado para evitar exploración indefinida.

A*. Emplea una cola de prioridad con f(n)=g(n)+h(n). La heurística pondera diferencias de día, hora y profesional con los mismos costos del modelo, orientando la búsqueda hacia alternativas próximas. Russell y Norvig ubican BFS, DFS y A* dentro del estudio de solución de problemas por búsqueda (2021).

4. Complejidad Big O

Algoritmo	Tiempo	Espacio	Observación
BFS	O(b^d)	O(b^d)	Completo; alto consumo de memoria
DFS	O(b^m)	O(bm)	Menor memoria; no óptimo
A*	O(b^d) en peor caso	O(b^d)	La heurística reduce expansión práctica

En la notación, b es el factor de ramificación, d la profundidad de la solución más cercana y m la profundidad máxima. El máximo teórico de 966 estados limita el dominio real, por lo que también puede expresarse el recorrido exhaustivo como O(|V|+|E|) para la representación explícita del grafo.

5. Benchmark reproducible

El benchmark se ejecutó 20 veces por algoritmo sobre el mismo escenario controlado: estado inicial (0,0,0), horizonte de 45 días, siete franjas, tres profesionales y metas (3,4,1), (5,2,2) y (7,6,0). La tabla presenta el promedio temporal y los conteos deterministas de la última ejecución.

Algoritmo	Expandidos	Generados	Tiempo medio (ms)	Costo
BFS	98	122	0.837705	11.5
DFS	393	852	3.927573	520.0
A*	40	75	0.343779	11.5

Los tiempos dependen del equipo y del entorno de ejecución, por lo que deben interpretarse junto con los nodos expandidos y generados. A* prioriza estados prometedores mediante la heurística; BFS mantiene una exploración por niveles; DFS puede alcanzar una meta con menos memoria, pero su resultado depende del orden de sucesores. El archivo benchmark.csv preserva los valores y el módulo src/search_algorithms.py permite repetir la prueba.

6. Validación y reproducibilidad

La prueba automatizada confirma que BFS, DFS y A* alcanzan una meta válida y expanden al menos un estado. Para validar la aplicación completa se deben probar cuatro casos: reprogramación y cancelación desde el portal, y reprogramación y cancelación desde Administración. Cada operación debe producir tres filas de análisis, actualizar el historial, enviar la notificación y limpiar el formulario solo después de una operación exitosa.

7. Conclusiones

La formalización permite convertir la agenda veterinaria en un problema de búsqueda reproducible. BFS ofrece una referencia completa por niveles; DFS reduce memoria, aunque no garantiza el mejor resultado; y A* utiliza conocimiento del dominio para orientar la exploración. La comparación no se limita al tiempo: nodos expandidos, nodos generados, costo y trazabilidad ofrecen una evaluación más sólida.

La arquitectura mantiene separadas la interfaz, la persistencia y la lógica de búsqueda. Esta separación facilita probar los algoritmos fuera de Colab y evita publicar información sensible. La persistencia permite conservar reservas entre sesiones, mientras que el historial documenta decisiones asociadas con cancelaciones y reprogramaciones.

Referencias

Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Project Jupyter. (2026). Project Jupyter documentation. https://jupyter.org/documentation

Python Software Foundation. (2026). Python 3 documentation. https://docs.python.org/3/

Enlace del repositorio

https://github.com/USUARIO/agente-citas-veterinarias-v7-3

Control final: el enlace anterior debe reemplazarse por la URL pública real antes de entregar. Verifique el enlace desde una ventana de incógnito y confirme que el notebook, el código fuente y el README sean visibles.